

Lab-10

Course: Compiler Design Laboratory

SLOT: L23+L24

Name: Shrri Dharshan D R

Reg No: 23BPS1090

OBJECTIVE

To study and implement three-address code representations like Quadruple, Triple, and Indirect Triple for a specific set of statements using a C program.

PROBLEM STATEMENT

Write a C program to implement Quadruple, Triple, and Indirect Triple representations for the following three-address statements.

1. $S1 := 4 * i$
2. $S2 := a[S1]$
3. $S3 := 4 * i$
4. $S4 := b[S3]$
5. $S5 := S2 * S4$
6. $S6 := prod + S5$
7. $prod := S6$
8. $S7 := i + 1$
9. $i := S7$
10. **if $i \leq 20$ goto 1**

AIM

To implement Three Address Code representations such as Quadruple, Triple, and Indirect Triple for the given statements using a C program. Also, to study their intermediate code generation in compiler design.

PROCEDURE

1. Start the program.
2. Define the given three-address statements.
3. Construct the **Quadruple representation** using Operator, Argument1, Argument2, and Result fields.

4. Construct the **Triple representation** where intermediate results are referred using instruction numbers.
5. Construct the **Indirect Triple representation** using a pointer table that refers to triple instructions.
6. Display all three representations using formatted output.
7. Compile the program using the C compiler.
8. Execute the program and verify the results.
9. Stop the program.

Question

Implement three address codes of quardable, triple and indirect triple of three address statements using C program in the following statement

```
1.S1:= 4 * i
2.S2:= a[S1]
3.S3:= 4 * i
4.S4:= b[S3]
5.S5:= s2 * S4
6.S6:= prod + S5
7.Prod:= s6
8.S7:= i+1
9.i := S7
10.if i<= 20 goto (1)
Stages in DAG Const
```

CODE:

```
#include <stdio.h>
```

```
int main() {
```

```
    // Quadruple Representation
```

```
    printf("QUADRUPLE REPRESENTATION\n");
```

```
    printf("Index\tOp\tArg1\tArg2\tResult\n");
```

```
    printf("1\t*\t4\ti\tS1\n");    // S1 := 4 * i
```

```
    printf("2\t[\t]\ta\tS1\tS2\n");    // S2 := a[S1]
```

```
    printf("3\t*\t4\ti\tS3\n");    // S3 := 4 * i
```

```
    printf("4\t[\t]\tb\tS3\tS4\n");    // S4 := b[S3]
```

```
    printf("5\t*\tS2\tS4\tS5\n");    // S5 := S2 * S4
```

```
    printf("6\t+\tprod\tS5\tS6\n");    // S6 := prod + S5
```

```
    printf("7\t:=\tS6\t-\tprod\n");    // prod := S6
```

```
    printf("8\t+\ti\t1\tS7\n");    // S7 := i + 1
```

```
    printf("9\t:=\tS7\t-\ti\n");    // i := S7
```

```
    printf("10\t<=\ti\t20\tgoto 1\n"); // if i <= 20 goto 1
```

```
    // Triple Representation
```

```
    printf("\nTRIPLE REPRESENTATION\n");
```

```
    printf("Index\tOp\tArg1\tArg2\n");
```

```
    printf("0\t*\t4\ti\n");
```

```
    printf("1\t[\t]\ta\t(0)\n");
```

```
    printf("2\t*\t4\ti\n");
```

```
    printf("3\t[\t]\tb\t(2)\n");
```

```
    printf("4\t*\t(1)\t(3)\n");
```

```
printf("5\t+\tprod\t(4)\n");
printf("6\t:=\t(5)\tprod\n");
printf("7\t+\ti\t1\n");
printf("8\t:=\t(7)\ti\n");
printf("9\t<=\ti\t20\n");
printf("goto 0\n");
```

```
// Indirect Triple Representation
```

```
printf("\nINDIRECT TRIPLE REPRESENTATION\n");
printf("Pointer Table\n");
```

```
printf("P0 -> 0\n");
printf("P1 -> 1\n");
printf("P2 -> 2\n");
printf("P3 -> 3\n");
printf("P4 -> 4\n");
printf("P5 -> 5\n");
printf("P6 -> 6\n");
printf("P7 -> 7\n");
printf("P8 -> 8\n");
printf("P9 -> 9\n");
```

```
return 0;
```

```
}
```

OUTPUT:

```
./a.out
QUADRUPLE REPRESENTATION
Index Op    Arg1  Arg2  Result
1      *      4      i      S1
2      []     a      S1      S2
3      *      4      i      S3
4      []     b      S3      S4
5      *      S2     S4      S5
6      +      prod   S5      S6
7      :=     S6      -      prod
8      +      i      1      S7
9      :=     S7      -      i
10     <=     i      20     goto 1

TRIPLE REPRESENTATION
Index Op    Arg1  Arg2
0      *      4      i
1      []     a      (0)
2      *      4      i
3      []     b      (2)
4      *      (1)   (3)
5      +      prod  (4)
6      :=     (5)   prod
7      +      i      1
8      :=     (7)   i
9      <=     i      20
goto 0

INDIRECT TRIPLE REPRESENTATION
Pointer Table
p0 -> 0
p1 -> 1
p2 -> 2
p3 -> 3
p4 -> 4
p5 -> 5
p6 -> 6
p7 -> 7
p8 -> 8
p9 -> 9
shrridharshan@SD:~$ |
```

RESULT

The Quadruple, Triple, and Indirect Triple representations for the three-address statements were successfully implemented and run using a C program.